

NoSQL

THERE IS more to data than tables

<Although NoSQL is hailed as a single fix for all database issues, some prefer good-old SQL. But, what is NoSQL?> **By Kshitij Sobti**

Is NoSQL the solution to all database woes? Are all those still sticking with SQL databases just old fashioned and unable to move on? As usual when a new technology emerges – or when it becomes popular – it isn't as simple as picking one or the other; so the answer to both these questions is a No. Both have use cases in which they excel, and each asks you to make certain compromises; how these align to the needs of the application, is what matters.

First of all, NoSQL isn't even new. It has been around for a long while. If you take NoSQL to strictly mean a database that doesn't use the SQL query language, you will find many old databases that can be operated without the use of SQL. There were databases before SQL, and those weren't accessed using SQL, of course. More frequently, NoSQL means non-relational. Before we talk of non-relational databases though, let's first take a look at what is a relational database.

Relational Databases

If a database is called relational, that is usually taken to mean that it stores data in the form of tables, where each table can refer to other tables. How would this store data though, you ask?

Let us consider a sample case – the classic example of students in a classroom. For each student, we have a name, and a roll number. Each student can enrol in any number of subjects, and for each subject there can be any number of examinations, each with a score. As you can see, such data is highly structured.

In a relational database, we would have a table that contains columns for the name of the student, and the roll number. In another table we would then have the student's roll number, the subject name, and an enrolment ID that is unique for each student and subject. Finally the exam scores could then be in a third table with columns for the enrolment id, and a score.

If we wanted to query all the scores of a particular student, we would search the table with the names for the student and look up the roll number. From the roll number we would check in the second table for the subjects the student is enrolled in, and the enrolment ID in each. Then for each enrolment ID, we would query the final scores table for the all the exam scores associated with that ID. We now have all the data associated with a particular student.

Now let's approach it differently. What if we need the names of all students who have failed? We query the exams table for scores below a threshold, and find out all the enrolment IDs. There might be duplicates here, since a student could have failed in multiple exams of the same subject, we can remove them, or not. From these enrolment IDs we can find out the particular subjects and roll numbers of students. Finally, the students names can be deduced from

the roll numbers – again there might be duplicates if a student has failed in multiple subjects.

Does each exam has multiple scores, such as internal and external assessments? Just add another table and expand the structure. Its quite flexible as you can see.

In most relational database systems, such queries would just be a single SQL statement! Common tasks such as removing duplicates, or finding aggregates etc. can also be accomplished in a query, rather than in the code later. Most such systems also enforce that all the data in a column be of a certain type. The name column for example, will be text, while the exam score column will only have numbers. This sounds good enough, what's the point of non-relational databases then? Let's have a look.

Non-Relational databases

First of all, non-relational databases are of many types, each with different characteristics. Not all of them will be suitable for all kinds of data.

Applications frequently store data in plain text files, which is another way to store data. Now, for an application storing its configuration settings, there is no need for complex querying of data, no sorting by columns, no aggregating data, and no need to eliminate duplicates. In such a case, a simple text file will be a lot more appropriate, and a lot more efficient. Of course this doesn't mean that a text file will be appropriate in the above scenario.

Spreadsheet software, such as the Microsoft Excel or LibreOffice Calc are also useful for storing data; however, storing the kind of data we discussed above is not feasible, or at least not efficient. We know what you're thinking, "but these aren't databases! Show me some databases." We hear you.

Small steps; let's take the example of the Windows registry. It is a central location that all applications can use to store data, so it is in fact a database. It was intended to do away with the need for having applications store configuration information in text files, or in their own formats – which makes it difficult to find, change, export and back up such information in bulk. The Windows registry stores settings in a simple format. Each setting has a name (or "key"), and a value.

How an application uses this is up to the creator. Values could be strings, numbers, binary data etc. All of these are contained in a hierarchy of folders. This keeps each applications settings separate, while still giving structure to the data. Complex operations such as searching for data are rarely required – each application knows exactly where its data is stored – so searching etc. while possible, is not optimised. There are other examples of hierarchical databases like the Windows Registry.